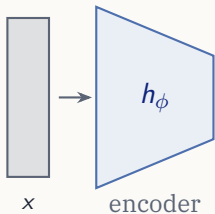


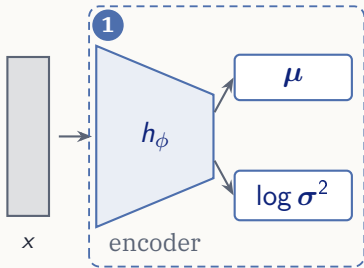
# The VAE in One Diagram

Member 1 showed the idea. Here are the same three stages with the symbols we use from now on.



# The VAE in One Diagram

Member 1 showed the idea. Here are the same three stages with the symbols we use from now on.



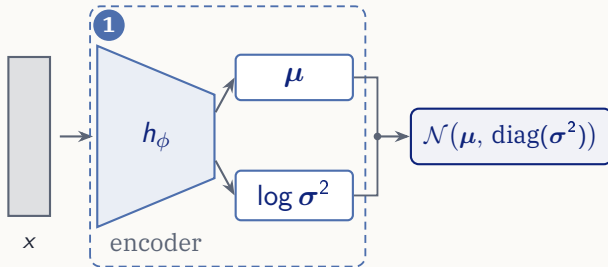
## 1 Encoder

$$x \mapsto (\mu, \log \sigma^2)$$

two vectors, not a point

# The VAE in One Diagram

Member 1 showed the idea. Here are the same three stages with the symbols we use from now on.



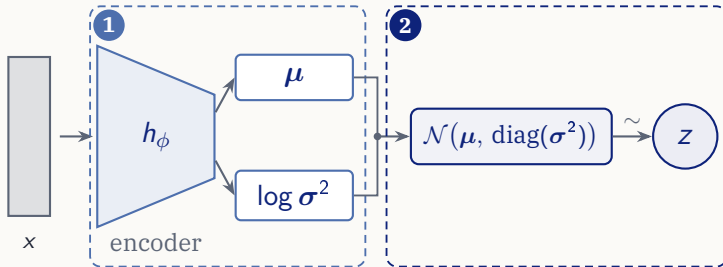
## 1 Encoder

$$x \mapsto (\mu, \log \sigma^2)$$

two vectors, not a point

# The VAE in One Diagram

Member 1 showed the idea. Here are the same three stages with the symbols we use from now on.



## 1 Encoder

$x \mapsto (\mu, \log \sigma^2)$

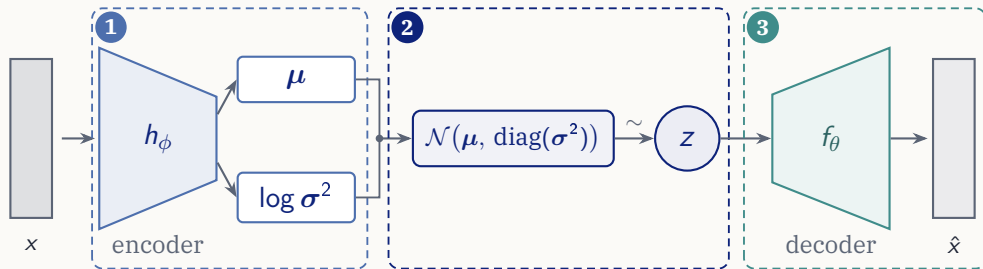
two vectors, not a point

## 2 Sampling

$z$  is drawn, not read off

# The VAE in One Diagram

Member 1 showed the idea. Here are the same three stages with the symbols we use from now on.



## 1 Encoder

$x \mapsto (\mu, \log \sigma^2)$

two vectors, not a point

## 2 Sampling

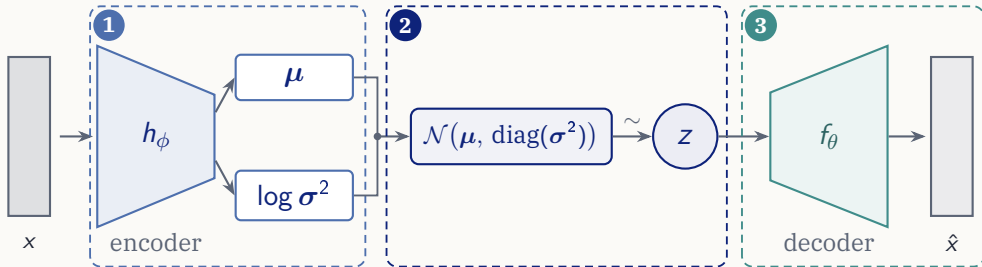
$z$  is drawn, not read off

## 3 Decoder

$z \mapsto \hat{x}$ , a full image again

# The VAE in One Diagram

Member 1 showed the idea. Here are the same three stages with the symbols we use from now on.



## 1 Encoder

$$x \mapsto (\mu, \log \sigma^2)$$

two vectors, not a point

## 2 Sampling

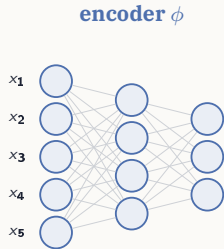
$z$  is drawn, not read off

## 3 Decoder

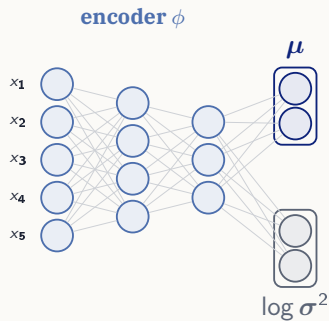
$$z \mapsto \hat{x}, \text{ a full image again}$$

Still to answer: why **two** heads? why **sampling** breaks training? where the **loss** comes from.

# The Encoder Outputs a Distribution



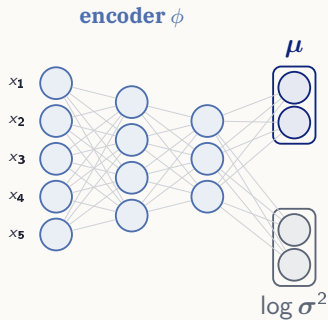
# The Encoder Outputs a Distribution



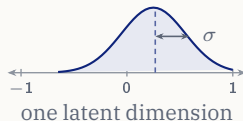
The same network as an autoencoder, except the bottleneck is split into **two** vectors — and two numbers per dimension *are* a curve:



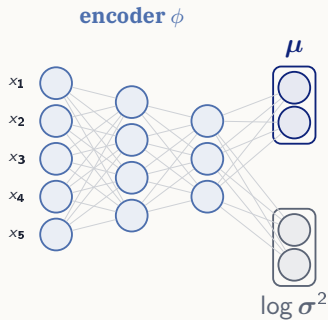
# The Encoder Outputs a Distribution



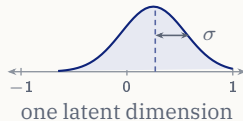
The same network as an autoencoder, except the bottleneck is split into **two** vectors — and two numbers per dimension *are* a curve:



# The Encoder Outputs a Distribution



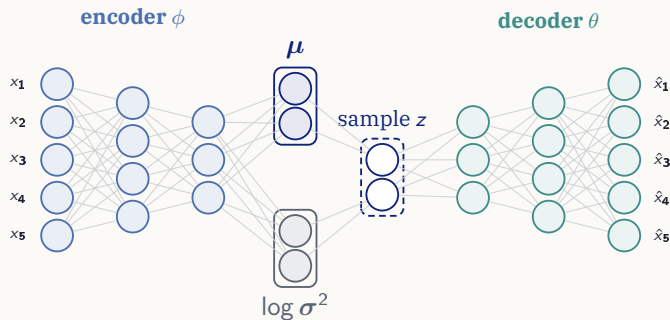
The same network as an autoencoder, except the bottleneck is split into **two** vectors — and two numbers per dimension *are* a curve:



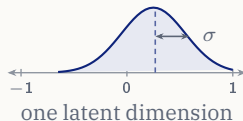
$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

Why **log** variance? A network output can be any real number, but  $\sigma$  must stay positive:  $\sigma = \exp(\frac{1}{2} \log \sigma^2) > 0$

# The Encoder Outputs a Distribution



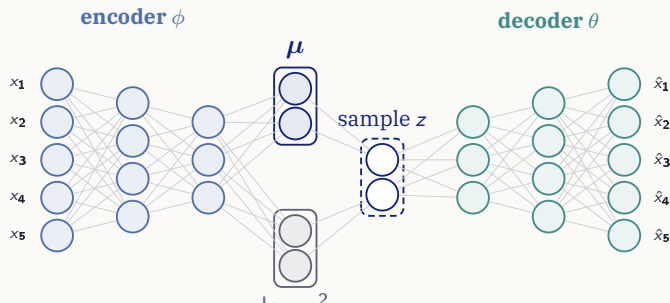
The same network as an autoencoder, except the bottleneck is split into **two** vectors — and two numbers per dimension *are* a curve:



$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

Why **log** variance? A network output can be any real number, but  $\sigma$  must stay positive:  $\sigma = \exp(\frac{1}{2} \log \sigma^2) > 0$

# The Encoder Outputs a Distribution



Every image now owns a small cloud in latent space — but the **sample** step is about to cause trouble.

The same network as an autoencoder, except the bottleneck is split into **two** vectors — and two numbers per dimension *are* a curve:

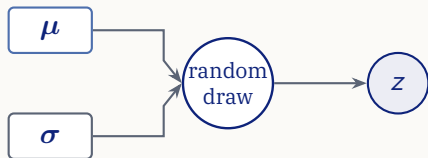


one latent dimension

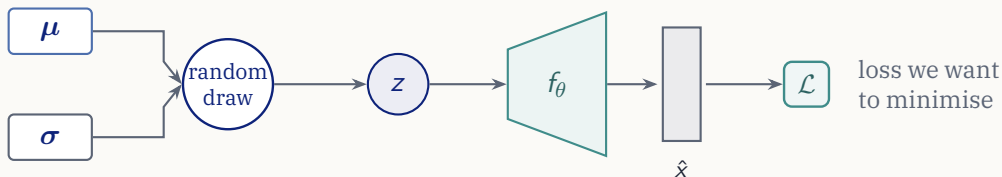
$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

Why **log** variance? A network output can be any real number, but  $\sigma$  must stay positive:  $\sigma = \exp(\frac{1}{2} \log \sigma^2) > 0$

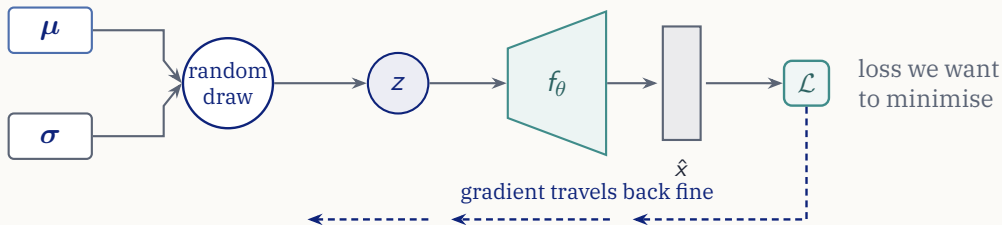
# Why Sampling Breaks Training



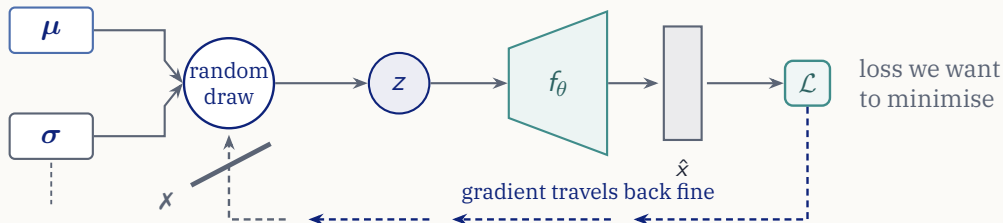
# Why Sampling Breaks Training



# Why Sampling Breaks Training



# Why Sampling Breaks Training



**No gradient reaches  $\mu$  or  $\sigma$ .**

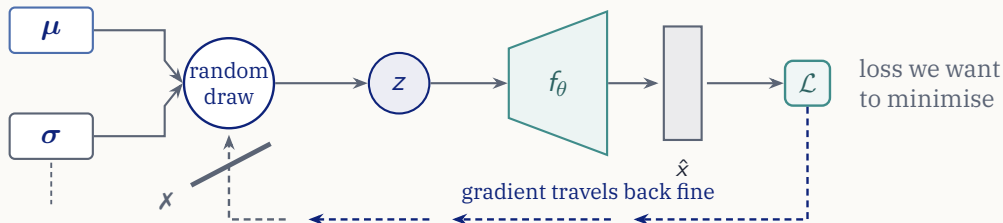
To train the encoder we need

$$\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial \phi}$$

but drawing a random number from  $\mathcal{N}(\mu, \sigma^2)$  is a **sampling step**, not a function of  $\mu, \sigma$ .



# Why Sampling Breaks Training



No gradient reaches  $\mu$  or  $\sigma$ .

To train the encoder we need

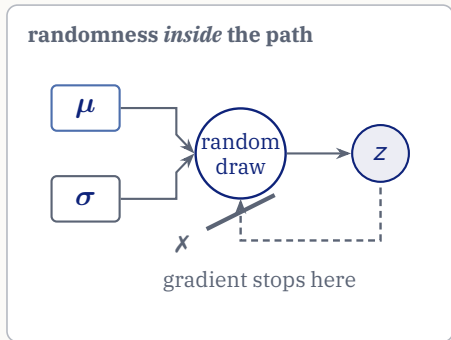
$$\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial \phi}$$

but drawing a random number from  $\mathcal{N}(\mu, \sigma^2)$  is a **sampling step**, not a function of  $\mu, \sigma$ .

$\frac{\partial z}{\partial \mu}$  and  $\frac{\partial z}{\partial \sigma}$  are **undefined** through a random draw.

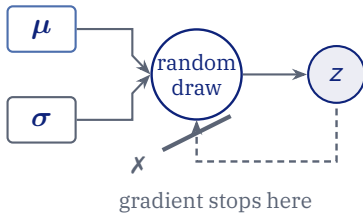
The decoder would still learn – the **encoder never would**.

# The Reparameterisation Trick

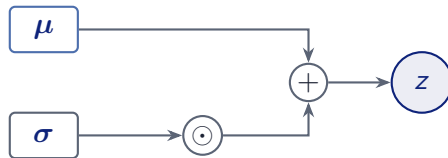


# The Reparameterisation Trick

randomness *inside* the path

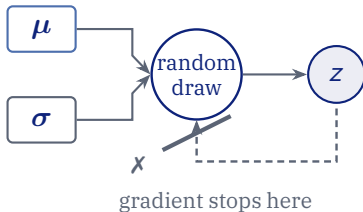


randomness moved *outside*

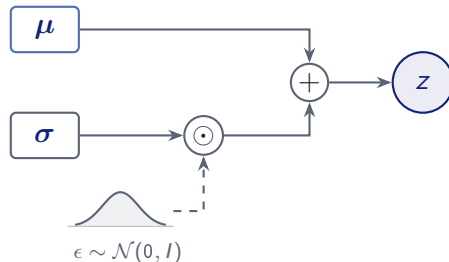


# The Reparameterisation Trick

randomness *inside* the path



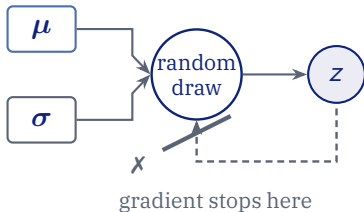
randomness moved *outside*



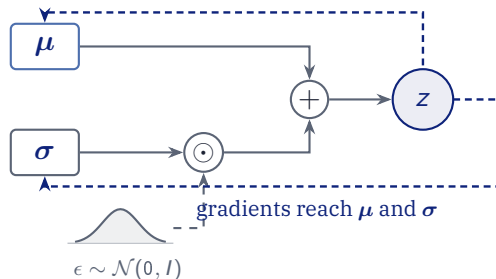
**Reparameterise:**  $z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

# The Reparameterisation Trick

randomness *inside* the path



randomness moved *outside*

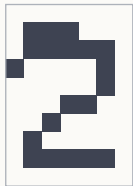


**Reparameterise:**  $z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

Now  $z$  is an ordinary function of  $\mu, \sigma, \epsilon$ :  $\partial z / \partial \mu = 1, \quad \partial z / \partial \sigma = \epsilon$  — backpropagation works.

# Loss Term 1: Reconstruction

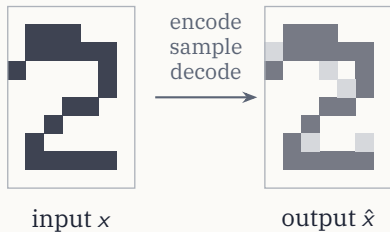
First job: whatever comes out must look like what went in.



input  $x$

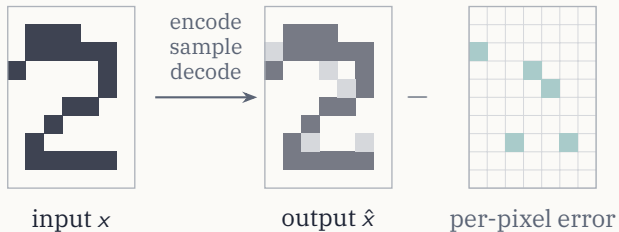
# Loss Term 1: Reconstruction

First job: whatever comes out must look like what went in.



# Loss Term 1: Reconstruction

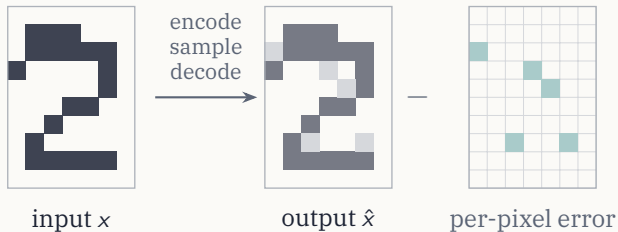
First job: whatever comes out must look like what went in.





# Loss Term 1: Reconstruction

First job: whatever comes out must look like what went in.

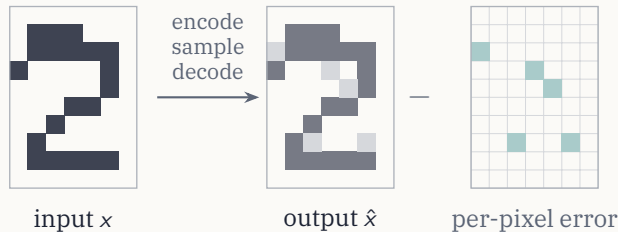


$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|^2 = \sum_i (x_i - \hat{x}_i)^2$$

Squared error for grey levels;  
**binary cross-entropy** if the  
pixels are 0/1.

# Loss Term 1: Reconstruction

First job: whatever comes out must look like what went in.



This term **alone** is just an autoencoder again: it would happily push every image onto its own tiny island and leave the rest of the space empty.

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|^2 = \sum_i (x_i - \hat{x}_i)^2$$

Squared error for grey levels;  
**binary cross-entropy** if the pixels are 0/1.

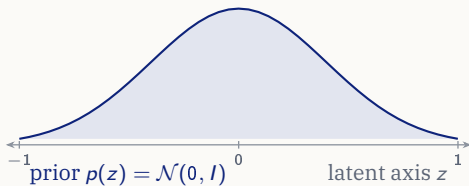
In probability form:

$$\mathcal{L}_{\text{recon}} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)]$$

i.e. make the decoder assign  
**high probability** to the image we actually fed in.

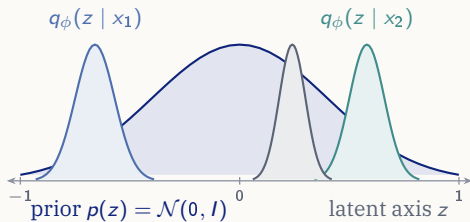
## Loss Term 2: the KL Pull

Second job: keep every one of those clouds **near the same fixed prior**.

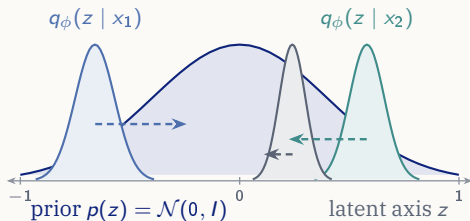


## Loss Term 2: the KL Pull

Second job: keep every one of those clouds **near the same fixed prior**.



# Loss Term 2: the KL Pull

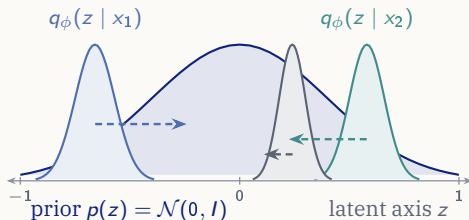


Second job: keep every one of those clouds **near the same fixed prior**.

$$\begin{aligned}\mathcal{L}_{\text{KL}} &= D_{\text{KL}}(q_\phi(z | x) \| p(z)) \\ &= \int q_\phi(z | x) \log \frac{q_\phi(z | x)}{p(z)} dz\end{aligned}$$

A distance between two distributions:  
0 when they match, larger as they drift  
apart.

## Loss Term 2: the KL Pull



The term pulls every  $\mu$  toward 0 and keeps every  $\sigma$  near 1, so each cloud stays **wide enough to overlap its neighbours** – that is what removes the gaps.

Second job: keep every one of those clouds **near the same fixed prior**.

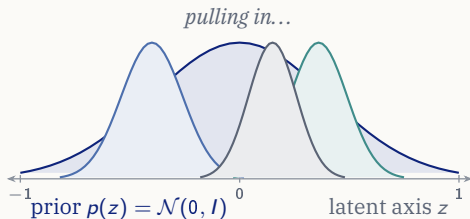
$$\begin{aligned}\mathcal{L}_{\text{KL}} &= D_{\text{KL}}(q_{\phi}(z | x) \| p(z)) \\ &= \int q_{\phi}(z | x) \log \frac{q_{\phi}(z | x)}{p(z)} dz\end{aligned}$$

A distance between two distributions:  
0 when they match, larger as they drift apart.

For diagonal Gaussians this is a **closed form** – no integral to compute:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

## Loss Term 2: the KL Pull



The term pulls every  $\mu$  toward 0 and keeps every  $\sigma$  near 1, so each cloud stays **wide enough to overlap its neighbours** – that is what removes the gaps.

Second job: keep every one of those clouds **near the same fixed prior**.

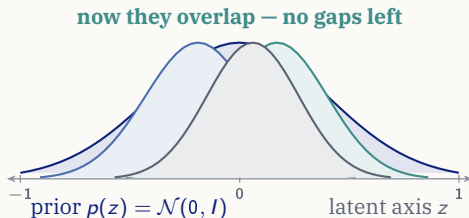
$$\begin{aligned}\mathcal{L}_{\text{KL}} &= D_{\text{KL}}(q_{\phi}(z | x) \parallel p(z)) \\ &= \int q_{\phi}(z | x) \log \frac{q_{\phi}(z | x)}{p(z)} dz\end{aligned}$$

A distance between two distributions:  
0 when they match, larger as they drift apart.

For diagonal Gaussians this is a **closed form** – no integral to compute:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

# Loss Term 2: the KL Pull



The term pulls every  $\mu$  toward 0 and keeps every  $\sigma$  near 1, so each cloud stays **wide enough to overlap its neighbours** – that is what removes the gaps.

Second job: keep every one of those clouds **near the same fixed prior**.

$$\begin{aligned}\mathcal{L}_{\text{KL}} &= D_{\text{KL}}(q_{\phi}(z | x) \parallel p(z)) \\ &= \int q_{\phi}(z | x) \log \frac{q_{\phi}(z | x)}{p(z)} dz\end{aligned}$$

A distance between two distributions:  
0 when they match, larger as they drift apart.

For diagonal Gaussians this is a **closed form** – no integral to compute:

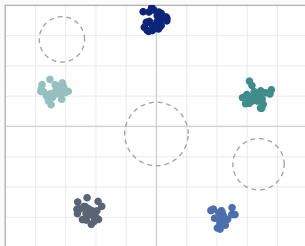
$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$



# What the Two Terms Do to the Space

Same data, same network, three different losses.

$\mathcal{L}_{\text{recon}}$  only

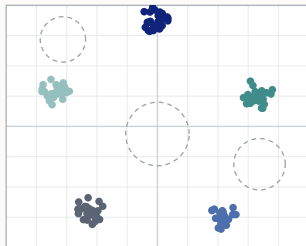


tight islands, wide  
**empty gaps** — decoding  
a gap gives nonsense

# What the Two Terms Do to the Space

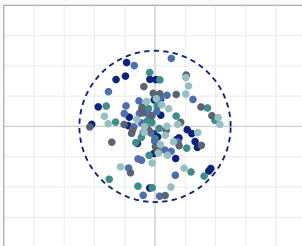
Same data, same network, three different losses.

$\mathcal{L}_{\text{recon}}$  only



tight islands, wide  
**empty gaps** — decoding  
a gap gives nonsense

$\mathcal{L}_{\text{KL}}$  only

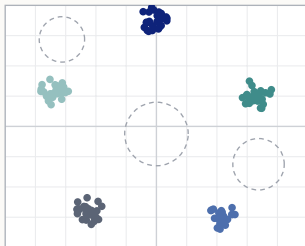


one blob, perfectly  
smooth — but every  $z$   
decodes to the **same blur**

# What the Two Terms Do to the Space

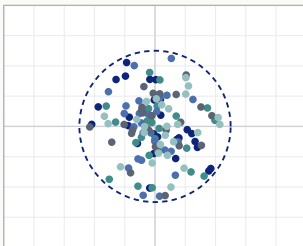
Same data, same network, three different losses.

$\mathcal{L}_{\text{recon}}$  only



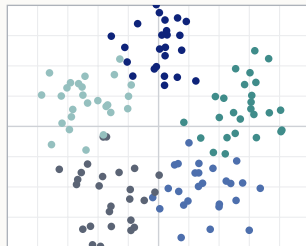
tight islands, wide  
**empty gaps** — decoding  
a gap gives nonsense

$\mathcal{L}_{\text{KL}}$  only



one blob, perfectly  
smooth — but every  $z$   
decodes to the **same blur**

both terms

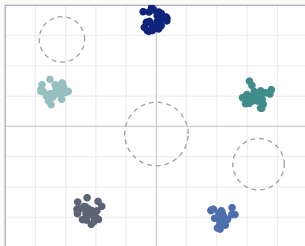


clusters that **touch**: separable,  
and no dead space left

# What the Two Terms Do to the Space

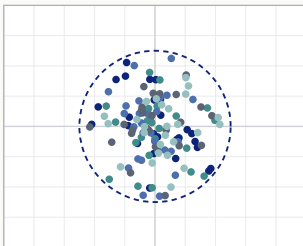
Same data, same network, three different losses.

$\mathcal{L}_{\text{recon}}$  only



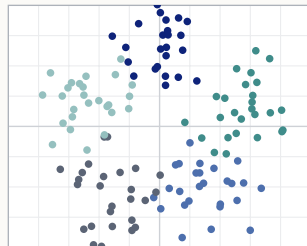
tight islands, wide  
**empty gaps** — decoding

$\mathcal{L}_{\text{KL}}$  only



one blob, perfectly  
smooth — but every  $z$

both terms



clusters that **touch**: separable,  
and no dead space left

$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}$  — the two terms pull against each other;  $\beta$  decides who wins

# What We Cannot Compute: the Bound Below It

What we would *like* to maximise:

$$\log p_{\theta}(x) = \log \int p_{\theta}(x | z) p(z) dz$$

✗ One integral over **every possible**  $z$ .  
Not computable — so we maximise a **lower bound** instead.

# What We Cannot Compute: the Bound Below It

What we would *like* to maximise:

$$\log p_{\theta}(x) = \log \int p_{\theta}(x | z) p(z) dz$$

✗ One integral over **every possible**  $z$ .  
Not computable — so we maximise a **lower bound** instead.

$$= \log p_{\theta}(x)$$

**ELBO**

$$\text{gap} \geq 0$$

# What We Cannot Compute: the Bound Below It

What we would *like* to maximise:

$$\log p_{\theta}(x) = \log \int p_{\theta}(x | z) p(z) dz$$

✗ One integral over **every possible**  $z$ .  
Not computable — so we maximise a **lower bound** instead.

$$= \log p_{\theta}(x)$$

**ELBO**

$$\text{gap} \geq 0$$

$$\text{gap} = D_{\text{KL}}(q_{\phi}(z | x) \| p_{\theta}(z | x))$$

The gap is another KL: how wrong the encoder's guess is. Push the **ELBO** up and  $\log p_{\theta}(x)$  has to come up with it — that is the whole trick.

# The Bound Is Exactly Our Two Terms

$$\text{ELBO} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(x | z)]}_{\text{page 5: reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z | x) \| p(z))}_{\text{page 6: KL pull}}$$



# The Bound Is Exactly Our Two Terms

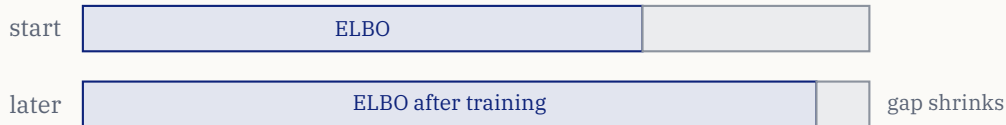
$$\text{ELBO} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(x | z)]}_{\text{page 5: reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z | x) \| p(z))}_{\text{page 6: KL pull}}$$

Nothing new to invent: the two terms we just built **are** the bound. And maximising it is the same as minimising the loss we train on,  $\mathcal{L}_{\text{VAE}} = -\text{ELBO}$ .

# The Bound Is Exactly Our Two Terms

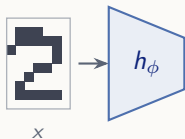
$$\text{ELBO} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(x | z)]}_{\text{page 5: reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z | x) \| p(z))}_{\text{page 6: KL pull}}$$

Nothing new to invent: the two terms we just built **are** the bound. And maximising it is the same as minimising the loss we train on,  $\mathcal{L}_{\text{VAE}} = -\text{ELBO}$ .

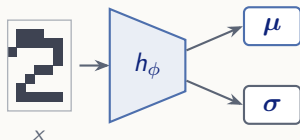


As  $q_\phi(z | x)$  gets closer to the truth the grey gap shrinks, the bound rises, and so does the real likelihood  $\log p_\theta(x)$ .

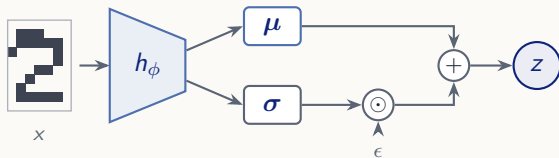
# One Training Step, End to End



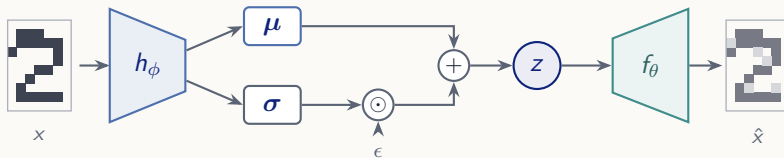
# One Training Step, End to End



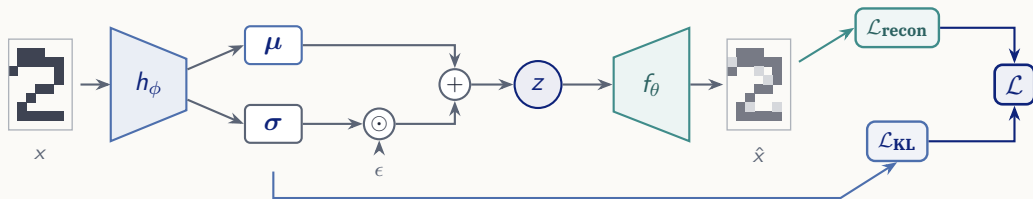
# One Training Step, End to End



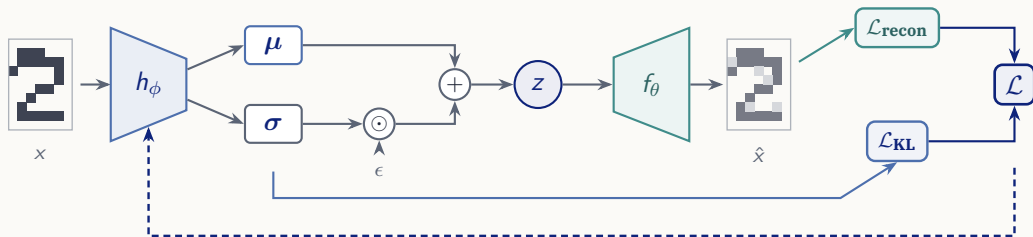
# One Training Step, End to End



# One Training Step, End to End



# One Training Step, End to End

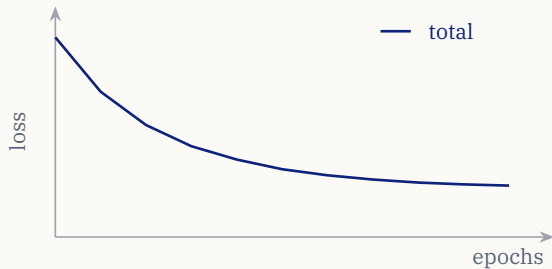


one backward pass updates  $\phi$  and  $\theta$  — it flows **through**  $\epsilon$

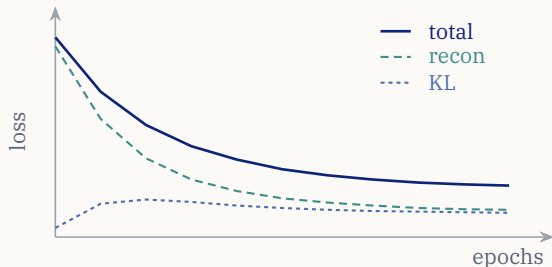
That is the entire training loop: one forward pass, one loss with two terms, one backward pass — an ordinary autoencoder plus  $\epsilon$ .



# What Training Looks Like

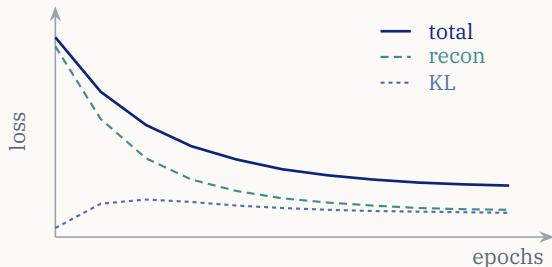


# What Training Looks Like



Reconstruction falls fast; the KL term rises a little at first — the two are trading against each other until they settle.

# What Training Looks Like



Reconstruction falls fast; the KL term rises a little at first — the two are trading against each other until they settle.

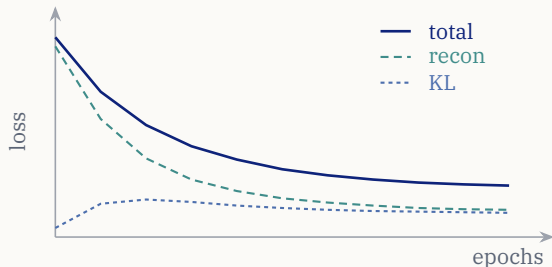
## Part 2 in three lines

$$q_{\phi}(z | x) = \mathcal{N}(\mu, \text{diag } \sigma^2)$$

$$z = \mu + \sigma \odot \epsilon$$

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}} = -\text{ELBO}$$

# What Training Looks Like



Reconstruction falls fast; the KL term rises a little at first — the two are trading against each other until they settle.

## Part 2 in three lines

$$q_{\phi}(z | x) = \mathcal{N}(\mu, \text{diag } \sigma^2)$$

$$z = \mu + \sigma \odot \epsilon$$

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}} = -\text{ELBO}$$

Over to **Member 3**: what a VAE can *do*.